UML Basics

The UML is a graphical language for visualizing, specifying, constructing, and documenting the artifacts of a software-intensive system. The UML gives you a standard way to write a system's blueprints.

The Importance of Modeling:

Model

A model is a simplification of reality. A model provides the blueprints of a system. A model may be structural, emphasizing the organization of the system, or it may be behavioral, emphasizing the dynamics of the system.

Why do we model

We build models so that we can better understand the system we are developing.

Through modeling, we achieve four aims.

1. Models help us to visualize a system as it is or as we want it to be.
2. Models permit us to specify the structure or behavior of a system.
3. Models give us a template that guides us in constructing a system.
4. Models document the decisions we have made.

We build models of complex systems because we cannot comprehend such a system in its entirety.

Applications of UML:-

The UML is intended primarily for software intensive system. It has been used effectively for such domains as

- Enterprise information system.
- Banking and financial services.
- Telecommunication.
- Defense/aerospace.
- Retail
- Medical electronics.
- Scientific.
- Distributed web based services.

The UML is not limited to modeling software. It is expressive enough to model non software system, such as workflow in the legal system, the structure and behaviors of a patient health care system, and the design of hardware.

Goals of UML:

The primary goals in the design of the UML were:

1. Provide users with a ready-to-use, expressive visual modeling language so they can develop and exchange meaningful models.
2. Provide extensibility and specialization mechanisms to extend the core concepts.

3. Be independent of particular programming languages and development processes.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of the OO tools market.
6. Support higher-level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.

Definition of Modeling:-

Modeling is overall design of the system.

- Modeling captures essential parts of the system.
- Modeling is a central part of all the activities that lead up to the deployment of good software.
- We build models to communication the desired structure and behavior of our system.
- We build models to visualize and controls the system's architecture
- We build models to better understanding the system we are building, after exposing opportunities for simplification and reuse.
- We build models to manage risk.

An Overview of UML

- The Unified Modeling Language is a standard language for writing software blueprints. The UML may be used to visualize, specify, construct, and document the artifacts of a software-intensive system.
- The UML is appropriate for modeling systems ranging from enterprise information systems to distributed Web-based applications and even to hard real time embedded systems. It is a very expressive language, addressing all the views needed to develop and then deploy such systems.

The UML is a language for

- Visualizing
- Specifying
- Constructing
- Documenting

- **Visualizing** The UML is more than just a bunch of graphical symbols. Rather, behind each symbol in the UML notation is a well-defined semantics. In this manner, one developer can write a model in the UML, and another developer, or even another tool, can interpret that model unambiguously
- **Specifying** means building models that are precise, unambiguous, and complete.
- **Constructing** the UML is not a visual programming language, but its models can be directly connected to a variety of programming languages
- **Documenting** a healthy software organization produces all sorts of artifacts in addition to raw executable code. These artifacts include

- Requirements
- Architecture
- Design
- Source code
- Project plans
- Tests
- Prototypes
- Releases

To understand the UML, you need to form a **conceptual model of the language**, and this requires learning three major elements:

1. Things
2. Relationships
3. Diagrams

Things in the UML

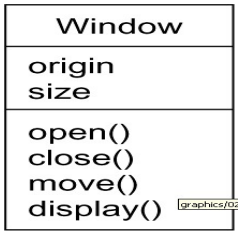
There are four kinds of things in the UML:

Structural things
Behavioral things
Grouping things
Annotational things

Structural things are the nouns of UML models. These are the mostly static parts of a model, representing elements that are either conceptual or physical. In all, there are seven kinds of structural things.

1. Classes
2. Interfaces
3. Collaborations
4. Use cases
5. Active classes
6. Components
7. Nodes

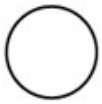
Class is a description of a set of objects that share the same attributes, operations, relationships, and semantics. A class implements one or more interfaces. Graphically, a class is rendered as a rectangle, usually including its name, attributes, and operations.



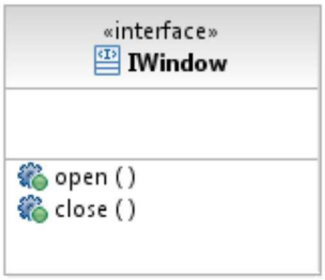
Interface

Interface is a collection of operations that specify a service of a class or component.
An interface therefore describes the externally visible behavior of that element.
An interface might represent the complete behavior of a class or component or only a part of that behavior.

An interface is rendered as a circle together with its name. An interface rarely stands alone. Rather, it is typically attached to the class or component that realizes the interface

IWindow 

or

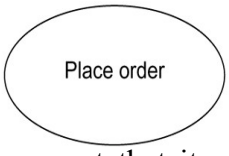


Collaboration defines an interaction and is a society of roles and other elements that work together to provide some cooperative behavior that's bigger than the sum of all the elements. Therefore, collaborations have structural, as well as behavioral, dimensions. A given class might participate in several collaborations.

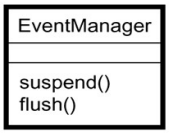
Graphically, a collaboration is rendered as an ellipse with dashed lines, usually including only its name

Usecase

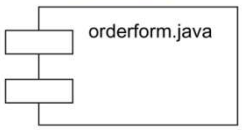
- Use case is a description of set of sequence of actions that a system performs that yields an observable result of value to a particular actor
- Use case is used to structure the behavioral things in a model.
- A use case is realized by a collaboration. Graphically, a use case is rendered as an ellipse with solid lines, usually including only its name



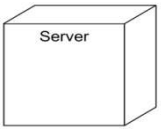
Active class is just like a class except that its objects represent elements whose behavior is concurrent with other elements. Graphically, an active class is rendered just like a class, but with heavy lines, usually including its name, attributes, and operations



Component is a physical and replaceable part of a system that conforms to and provides the realization of a set of interfaces. Graphically, a component is rendered as a rectangle with tabs



Node is a physical element that exists at run time and represents a computational resource, generally having at least some memory and, often, processing capability. Graphically, a node is rendered as a cube, usually including only its name



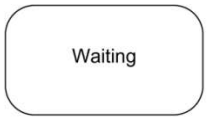
Behavioral Things are the dynamic parts of UML models. These are the verbs of a model, representing behavior over time and space. In all, there are two primary kinds of behavioral things
Interaction
state machine

Interaction

Interaction is a behavior that comprises a set of messages exchanged among a set of objects within a particular context to accomplish a specific purpose
An interaction involves a number of other elements, including messages, action sequences and links
Graphically a message is rendered as a directed line, almost always including the name of its operation

State Machine

State machine is a behavior that specifies the sequences of states an object or an interaction goes through during its lifetime in response to events, together with its responses to those events
State machine involves a number of other elements, including states, transitions, events and activities
Graphically, a state is rendered as a rounded rectangle, usually including its name and its substates.



Grouping Things:-

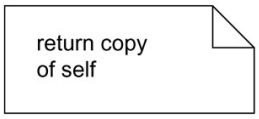
1. are the organizational parts of UML models. These are the boxes into which a model can be decomposed
2. There is one primary kind of grouping thing, namely, packages.

Package:-

- A package is a general-purpose mechanism for organizing elements into groups. Structural things, behavioral things, and even other grouping things may be placed in a package
- Graphically, a package is rendered as a tabbed folder, usually including only its name and, sometimes, its contents

Annotational things are the explanatory parts of UML models. These are the comments you may apply to describe about any element in a model.

A note is simply a symbol for rendering constraints and comments attached to an element or a collection of elements.
Graphically, a note is rendered as a rectangle with a dog-eared corner, together with a textual or graphical comment

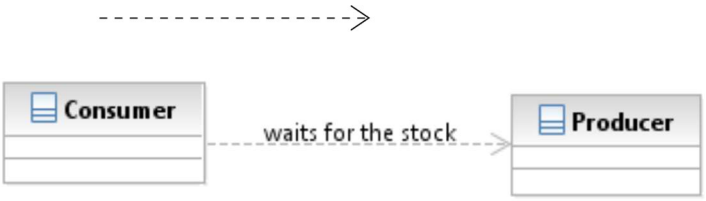


Relationships in the UML: There are four kinds of relationships in the UML:

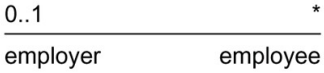
1. Dependency
2. Association
3. Generalization
4. Realization

Dependency:-
Dependency is a semantic relationship between two things in which a change to one thing may affect the semantics of the other thing
Graphically a dependency is rendered as a dashed line, possibly directed, and occasionally including a label

Eg

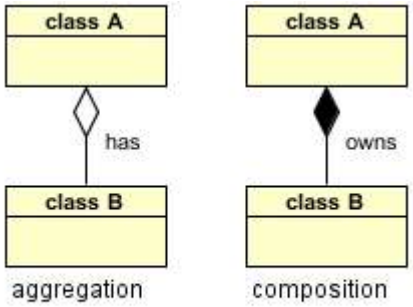


Association is a structural relationship that describes a set of links, a link being a connection among objects.
Graphically an association is rendered as a solid line, possibly directed, occasionally including a label, and often containing other adornments, such as multiplicity and role names

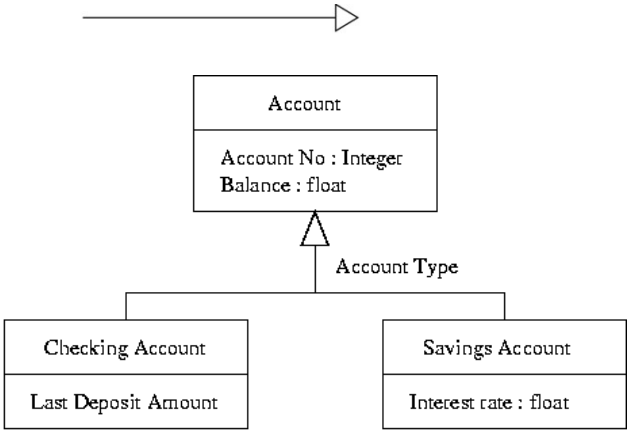


Aggregation implies a relationship where the child can exist independently of the parent. Example: Class (parent) and Student (child). Delete the Class and the Students still exist.

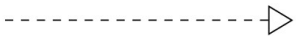
Composition implies a relationship where the child cannot exist independent of the parent. Example: House (parent) and Room (child). Rooms don't exist separate to a House.

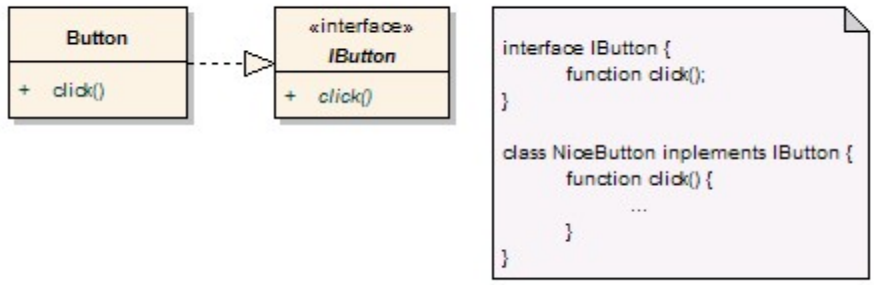


Generalization : **UML generalization** is a binary taxonomic relationship between a more general classifier (superclass) and a more specific classifier (subclass).Graphically, a generalization relationship is rendered as a solid line with a hollow arrowhead pointing to the parent



Realization is a semantic relationship between classifiers, wherein one classifier specifies a contract that another classifier guarantees to carry out. Graphically a realization relationship is rendered as a cross between a generalization and a dependency relationship





Diagrams in the UML

- **Diagram** is the graphical presentation of a set of elements, most often rendered as a connected graph of vertices (things) and arcs (relationships).
- In theory, a diagram may contain any combination of things and relationships.
- For this reason, the UML includes nine such diagrams:
    - Class diagram
    - Object diagram
    - Use case diagram
    - Sequence diagram
    - Communication diagram
    - Statechart diagram
    - Activity diagram
    - Component diagram
    - Deployment diagram

Class diagram

A class diagram shows a set of classes, interfaces, and collaborations and their relationships.
Class diagrams that include active classes address the static process view of a system.

Object diagram

- Object diagrams represent static snapshots of instances of the things found in class diagrams
- These diagrams address the static design view or static process view of a system
- An object diagram shows a set of objects and their relationships

Use case diagram

- A use case diagram shows a set of use cases and actors and their relationships
- Use case diagrams address the static use case view of a system.
- These diagrams are especially important in organizing and modeling the behaviors of a system.

Interaction Diagrams

Both sequence diagrams and collaboration diagrams are kinds of interaction diagrams
Interaction diagrams address the dynamic view of a system
A sequence diagram is an interaction diagram that emphasizes the time-ordering of messages

A communication diagram is an interaction diagram that emphasizes the structural organization of the objects that send and receive messages
Sequence diagrams and collaboration diagrams are isomorphic, meaning that you can take one and transform it into the other

Statechart diagram

- A statechart diagram shows a state machine, consisting of states, transitions, events, and activities
- Statechart diagrams address the dynamic view of a system
- They are especially important in modeling the behavior of an interface, class, or collaboration and emphasize the event-ordered behavior of an object

Activity diagram

An activity diagram is a special kind of a statechart diagram that shows the flow from activity to activity within a system
Activity diagrams address the dynamic view of a system
They are especially important in modeling the function of a system and emphasize the flow of control among objects

Component diagram

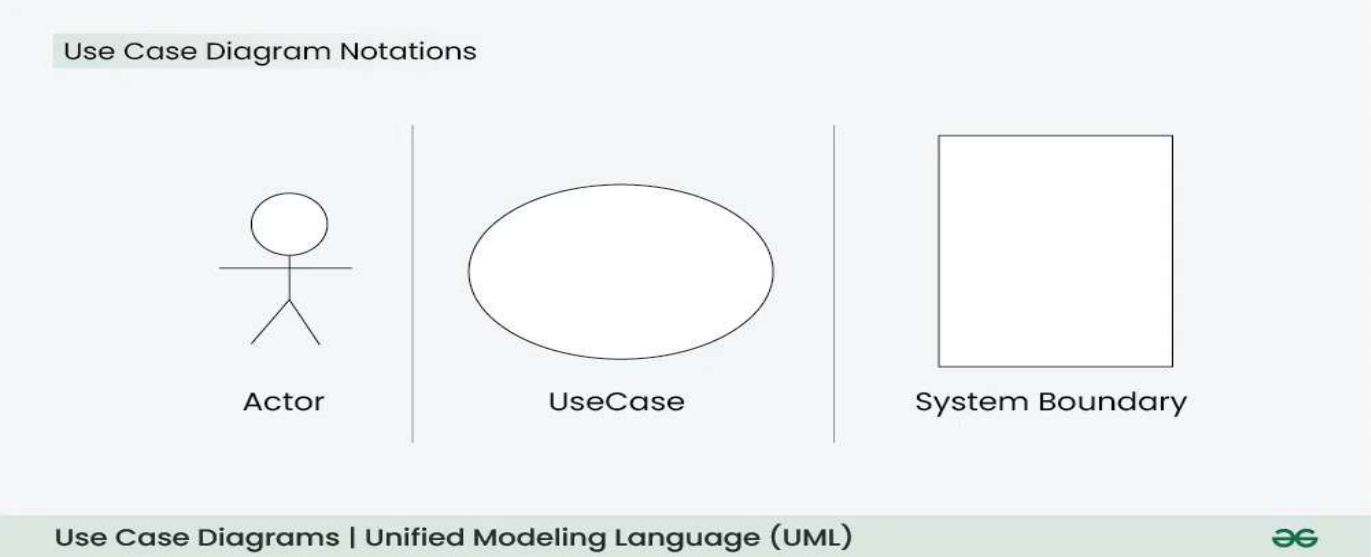
- A component diagram shows the organizations and dependencies among a set of components.
- Component diagrams address the static implementation view of a system
- They are related to class diagrams in that a component typically maps to one or more classes, interfaces, or collaborations

Deployment diagram

- A deployment diagram shows the configuration of run-time processing nodes and the components that live on them
- Deployment diagrams address the static deployment view of an architecture

What is a Use Case Diagram in UML?

A Use Case Diagram is a type of Unified Modeling Language (UML) diagram that represents the interaction between actors (users or external systems) and a system under consideration to accomplish specific goals. It provides a high-level view of the system's functionality by illustrating how users interact with it.



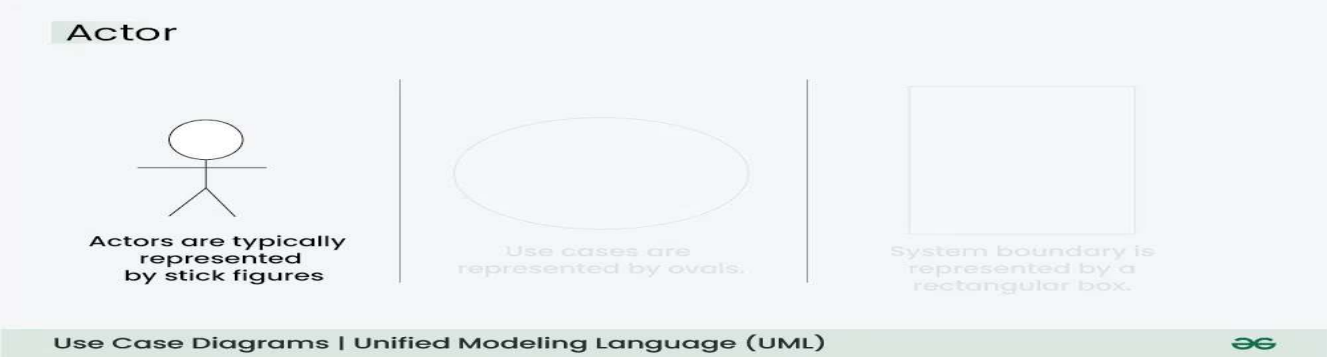
For more insights on creating effective Use Case Diagrams, the System Design Course covers the basics and provides practical applications in system design.

Use Case Diagram Notations

UML notations provide a visual language that enables software developers, designers, and other stakeholders to communicate and document system designs, architectures, and behaviors consistently and understandably.

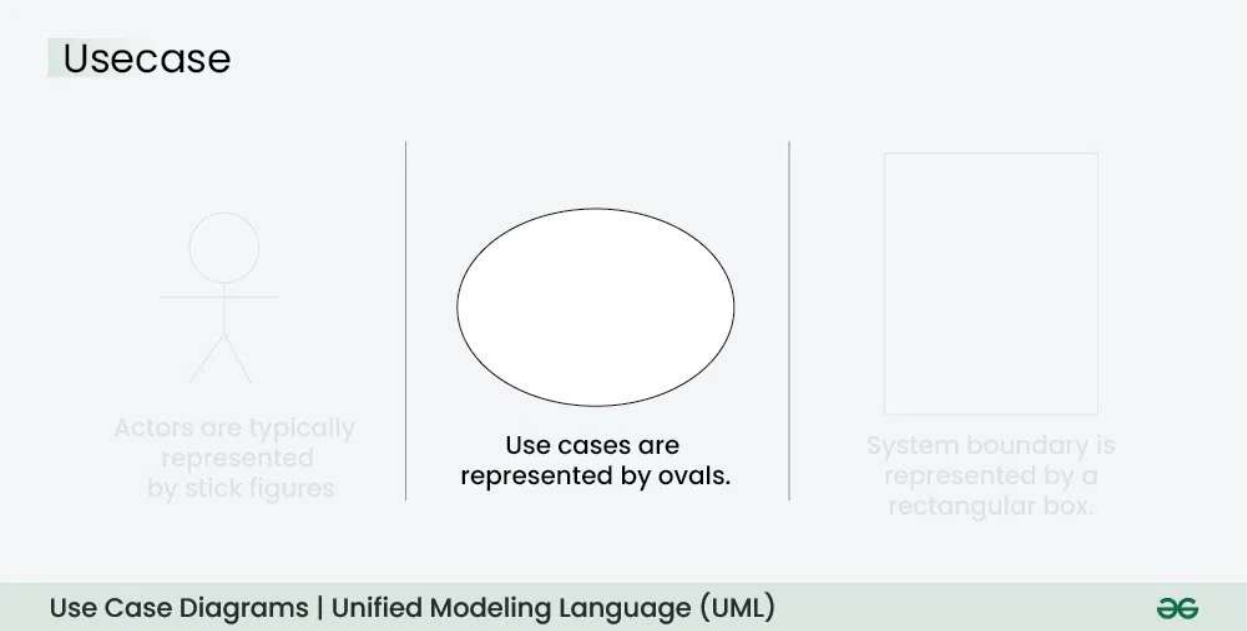
1. Actors

Actors are external entities that interact with the system. These can include users, other systems, or hardware devices. In the context of a Use Case Diagram, actors initiate use cases and receive the outcomes. Proper identification and understanding of actors are crucial for accurately modeling system behavior.



2. Use Cases

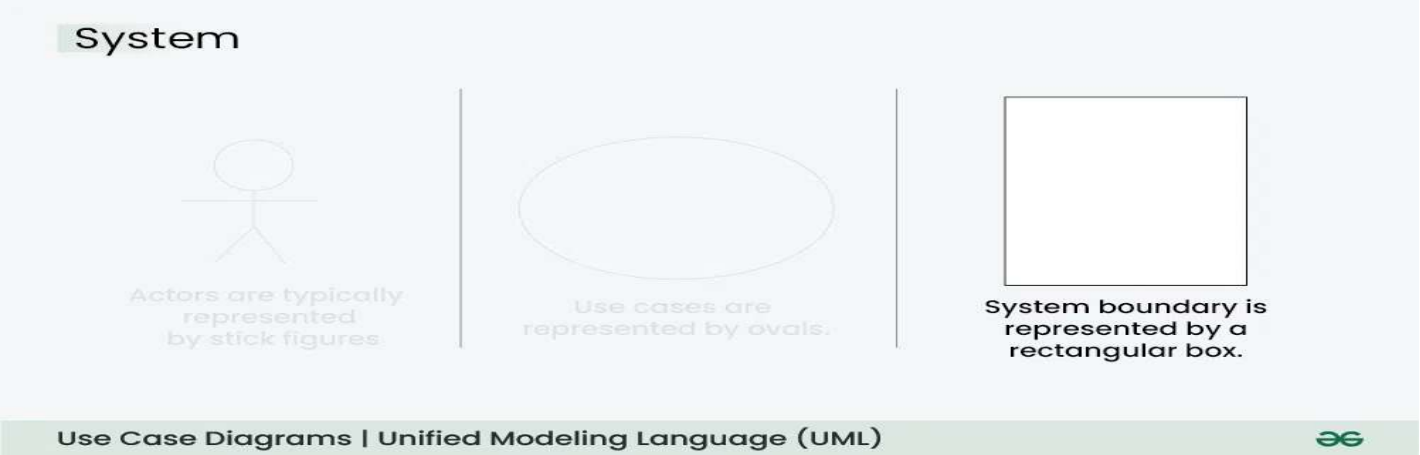
Use cases are like scenes in the play. They represent specific things your system can do. In the online shopping system, examples of use cases could be “Place Order,” “Track Delivery,” or “Update Product Information”. Use cases are represented by ovals.



3. System Boundary

The system boundary is a visual representation of the scope or limits of the system you are modeling. It defines what is inside the system and

what is outside. The boundary helps to establish a clear distinction between the elements that are part of the system and those that are external to it. The system boundary is typically represented by a rectangular box that surrounds all the use cases of the system.



Purpose of System Boundary:

- **Scope Definition:** It clearly outlines the boundaries of the system, indicating which components are internal to the system and which are external actors or entities interacting with the system.
- **Focus on Relevance:** By delineating the system's scope, the diagram can focus on illustrating the essential functionalities provided by the system without unnecessary details about external entities.

Use Case Diagram Relationships

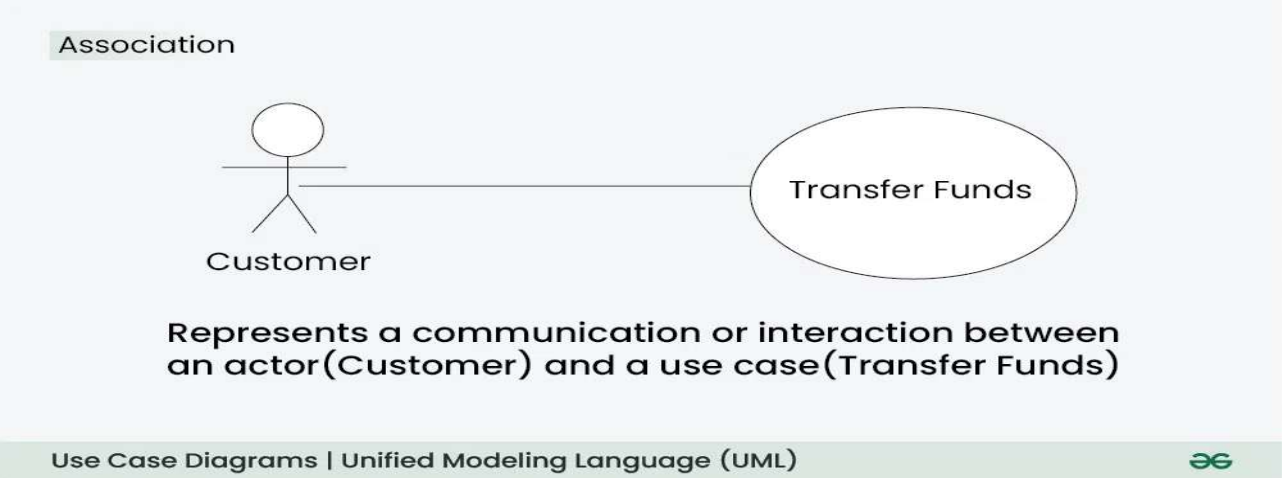
In a Use Case Diagram, relationships play a crucial role in depicting the interactions between actors and use cases. These relationships provide a comprehensive view of the system's functionality and its various scenarios. Let's delve into the key types of relationships and explore examples to illustrate their usage.

1. Association Relationship

The Association Relationship represents a communication or interaction between an actor and a use case. It is depicted by a line connecting the actor to the use case. This relationship signifies that the actor is involved in the functionality described by the use case.

Example: Online Banking System

- **Actor:** Customer
- **Use Case:** Transfer Funds
- **Association:** A line connecting the "Customer" actor to the "Transfer Funds" use case, indicating the customer's involvement in the funds transfer process.

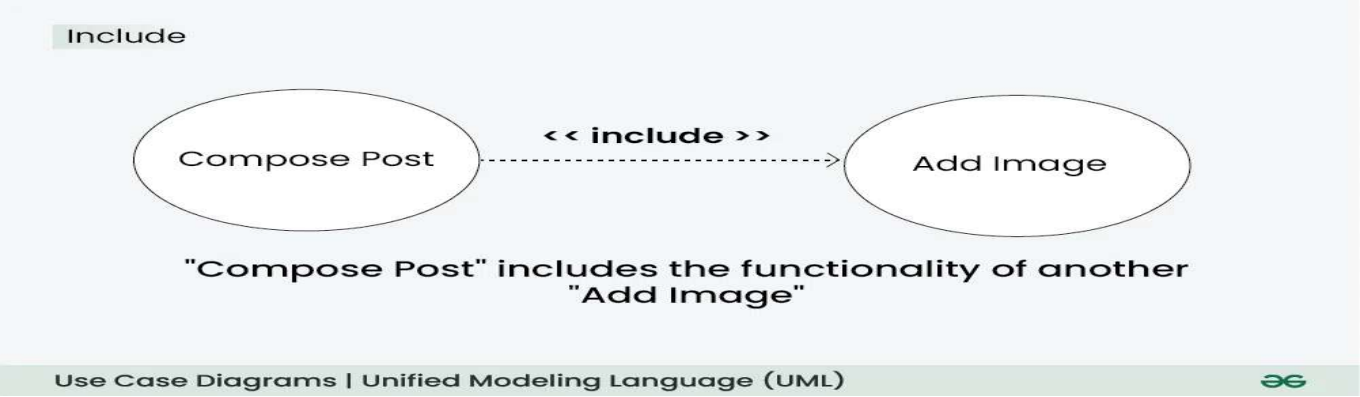


2. Include Relationship

The Include Relationship indicates that a use case includes the functionality of another use case. It is denoted by a dashed arrow pointing from the including use case to the included use case. This relationship promotes modular and reusable design.

Example: Social Media Posting

- **Use Cases:** Compose Post, Add Image
- **Include Relationship:** The "Compose Post" use case includes the functionality of "Add Image." Therefore, composing a post includes the action of adding an image.

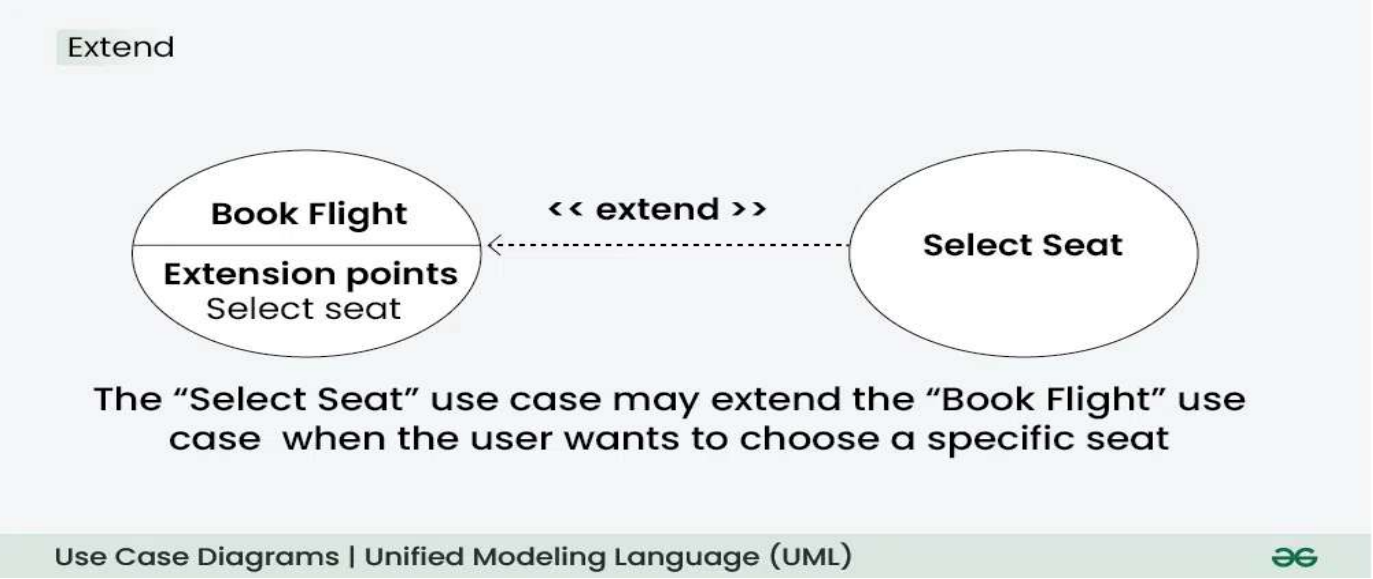


3. Extend Relationship

The Extend Relationship illustrates that a use case can be extended by another use case under specific conditions. It is represented by a dashed arrow with the keyword “extend.” This relationship is useful for handling optional or exceptional behavior.

Example: Flight Booking System

- **Use Cases:** Book Flight, Select Seat
- **Extend Relationship:** The “Select Seat” use case may extend the “Book Flight” use case when the user wants to choose a specific seat, but it is an optional step.

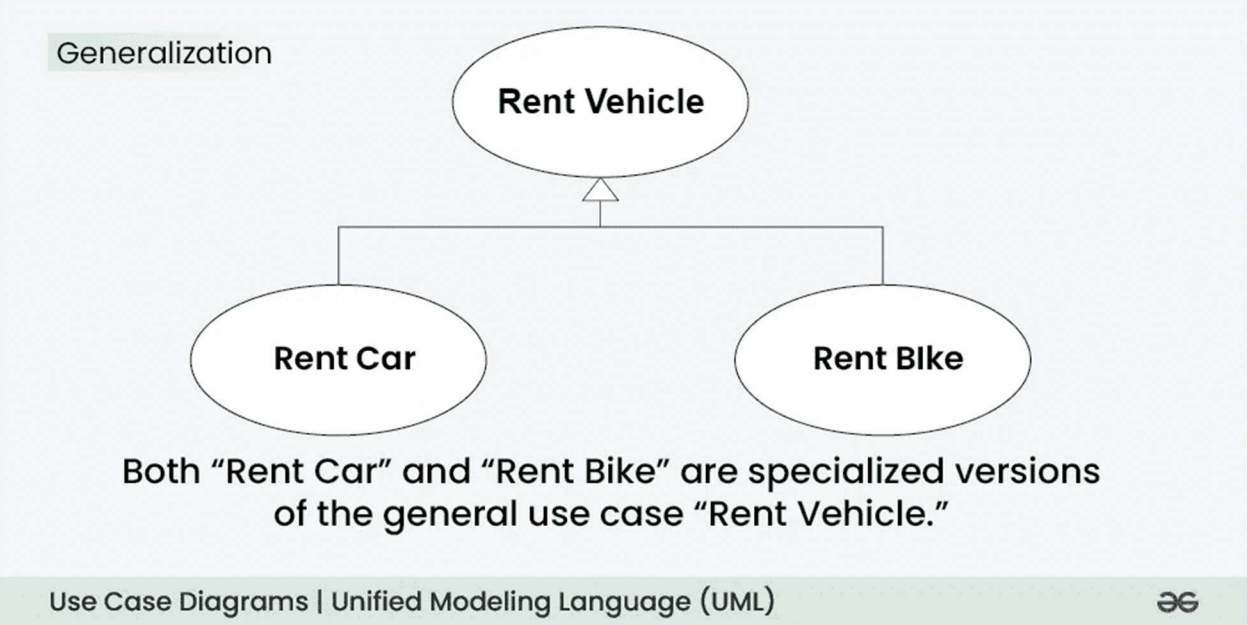


4. Generalization Relationship

The Generalization Relationship establishes an “is-a” connection between two use cases, indicating that one use case is a specialized version of another. It is represented by an arrow pointing from the specialized use case to the general use case.

Example: Vehicle Rental System

- **Use Cases:** Rent Car, Rent Bike
- **Generalization Relationship:** Both “Rent Car” and “Rent Bike” are specialized versions of the general use case “Rent Vehicle.”



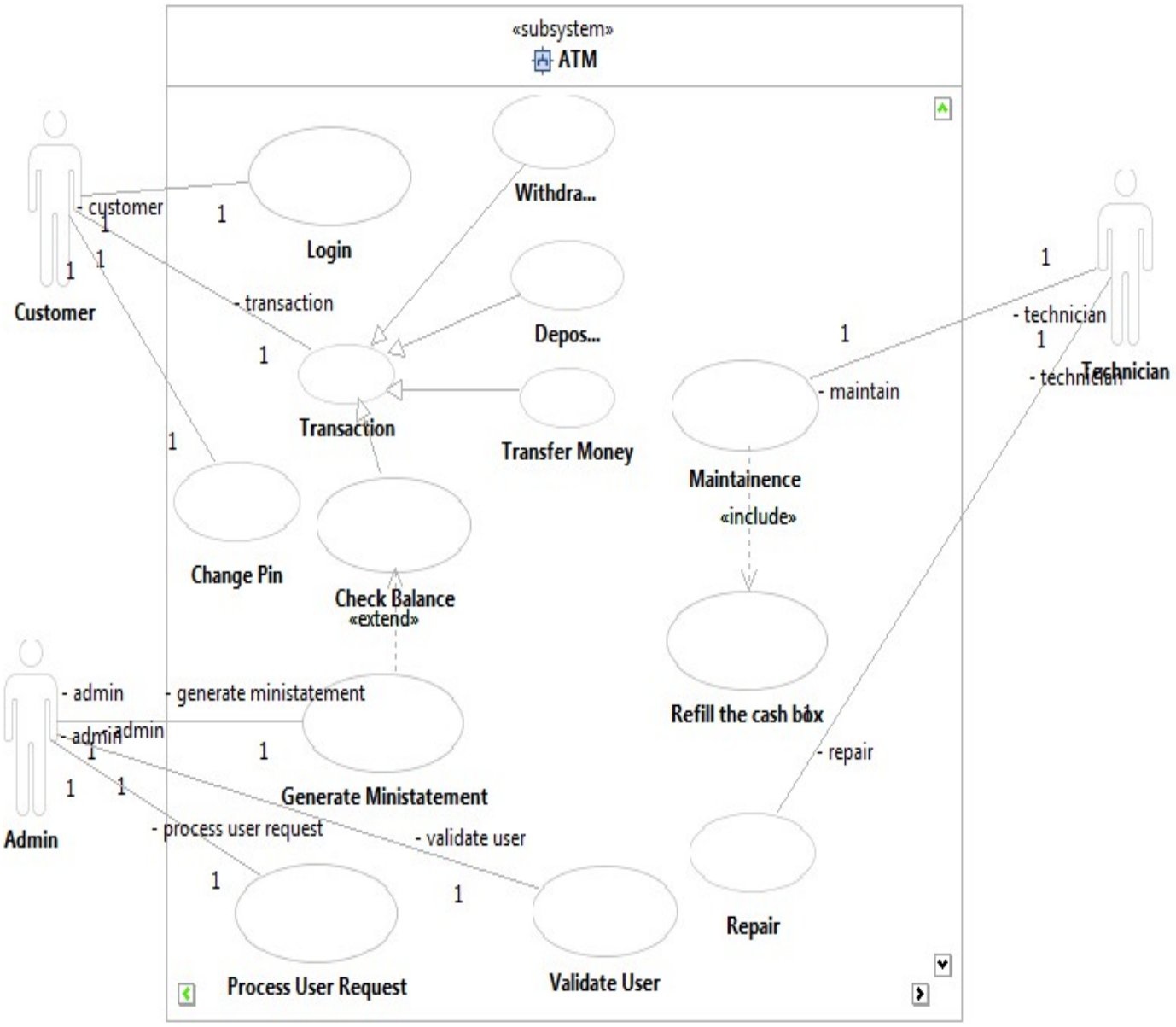
Generalization Relationship

How to Create a Use Case Diagram

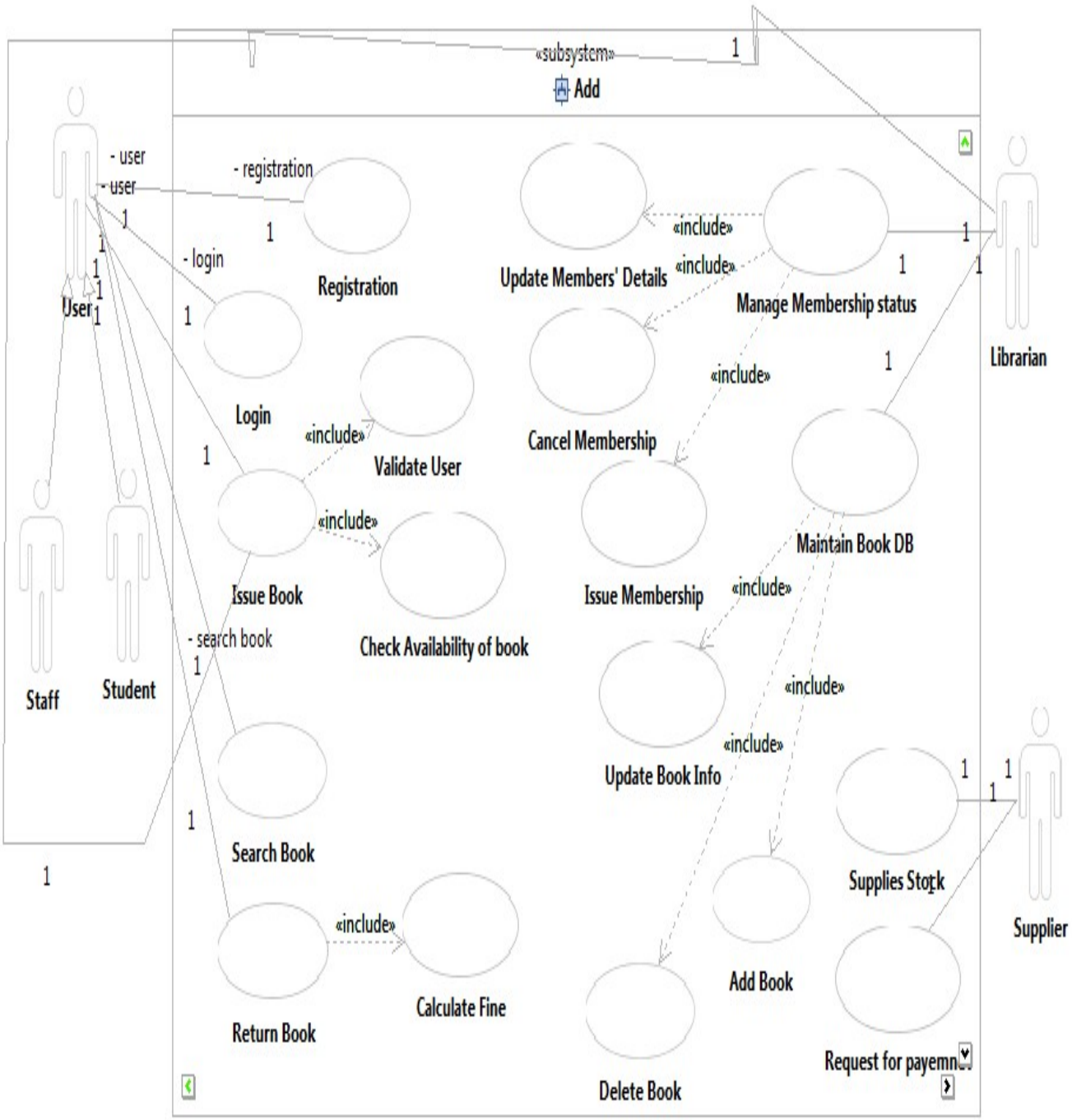
Up to now, you've learned about objects, relationships, and critical guidelines when drawing use case diagrams. I'll explain the various processes using a banking system as an example.

1. Identifying Actors
2. Identifying Use Cases
3. Look for Common Functionality to Reuse
4. Is it Possible to Generalize Actors and Use Cases
5. Optional Functions or Additional Functions
6. Validate and Refine the Diagram

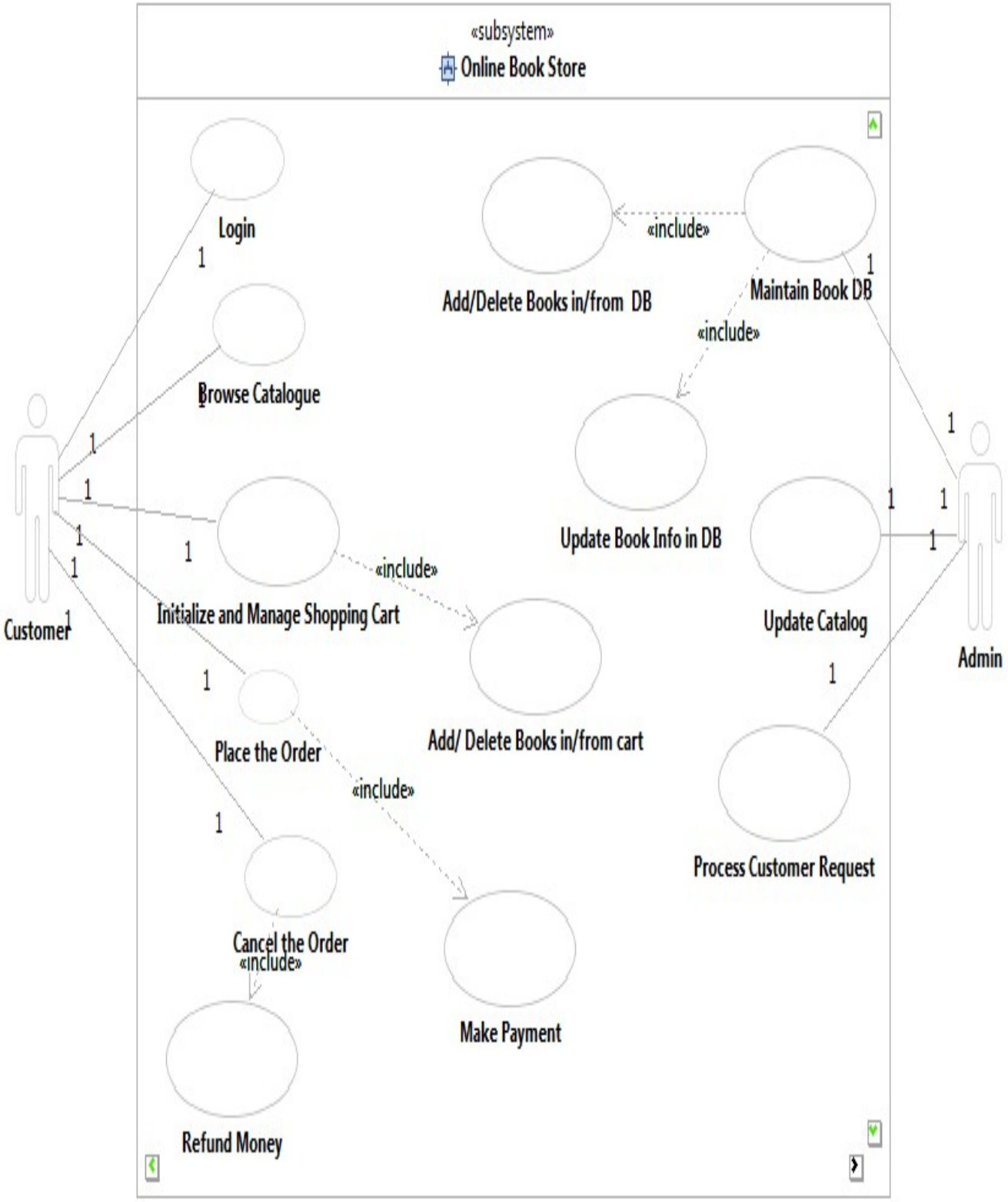
Draw Usecase Diagram for ATM System



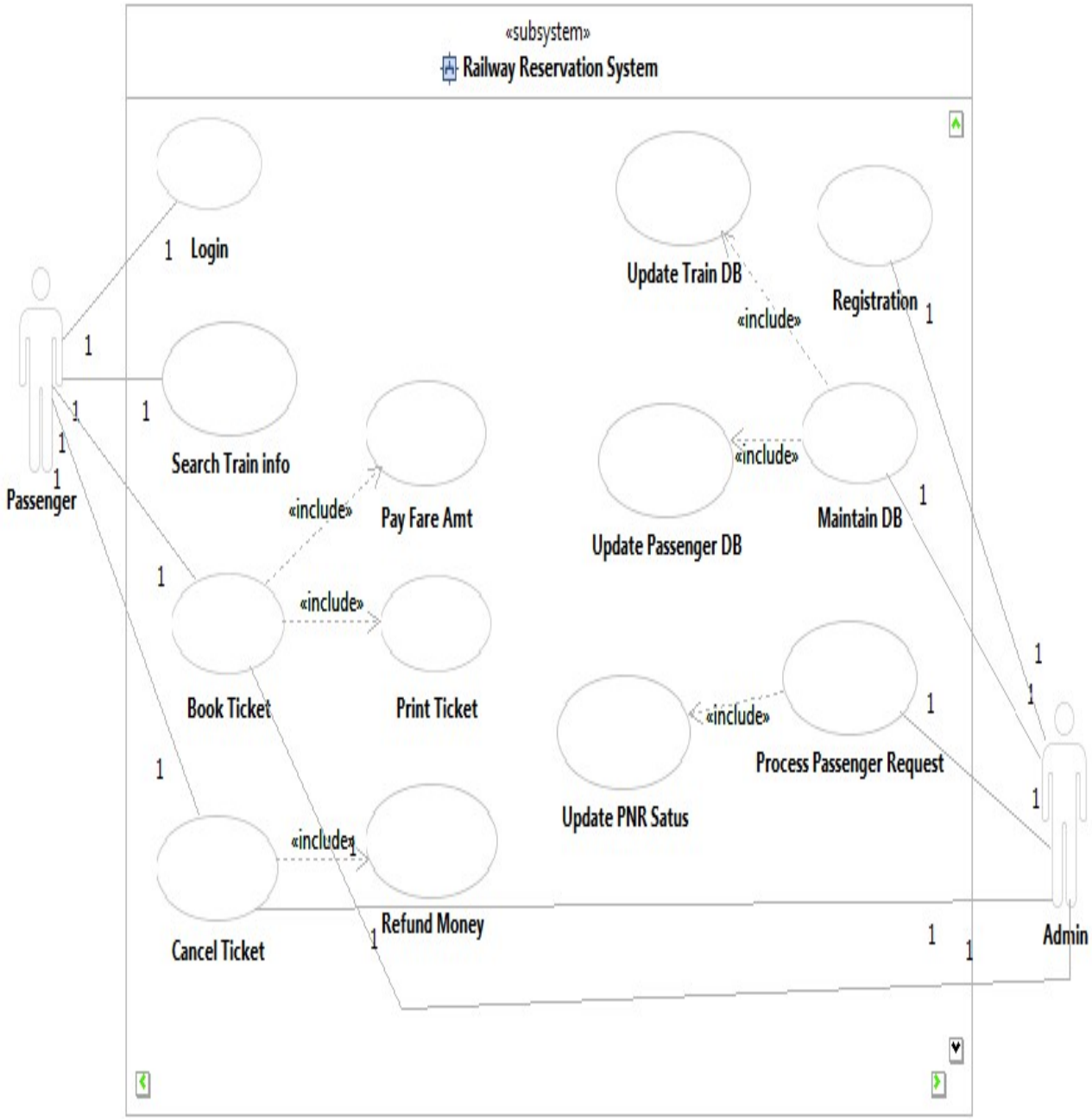
Draw Usecase Diagram for Library Management System


«subsystem»
Add
User
Staff
Student
Librarian
Supplier
- user
- user
- registration
- login
- search book
Registration
Login
Validate User
Issue Book
Check Availability of book
Search Book
Return Book
Calculate Fine
Update Members' Details
Manage Membership status
Cancel Membership
Issue Membership
Maintain Book DB
Update Book Info
Add Book
Delete Book
Supplies Stock
Request for payemn
«include»
«include»
«include»
«include»
«include»
«include»
«include»
«include»
«include»
1

Draw Usecase Diagram for Online Book shopping System



Draw Usecase Diagram for Online Train Ticket Reservation System

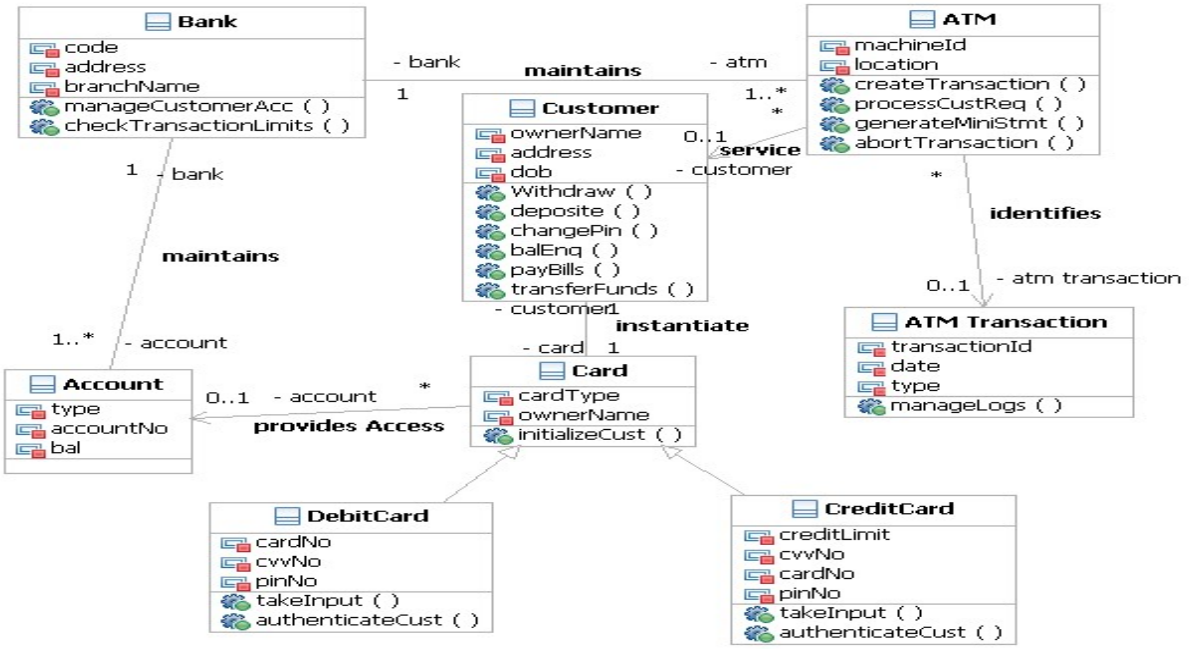


A **Class Diagram** in **UML (Unified Modeling Language)** is a type of static structure diagram that describes the structure of a system by showing its **classes**, **attributes**, **methods (or operations)**, and the **relationships** between the classes. Class diagrams are widely used in object-oriented design and provide a blueprint for the system's design.

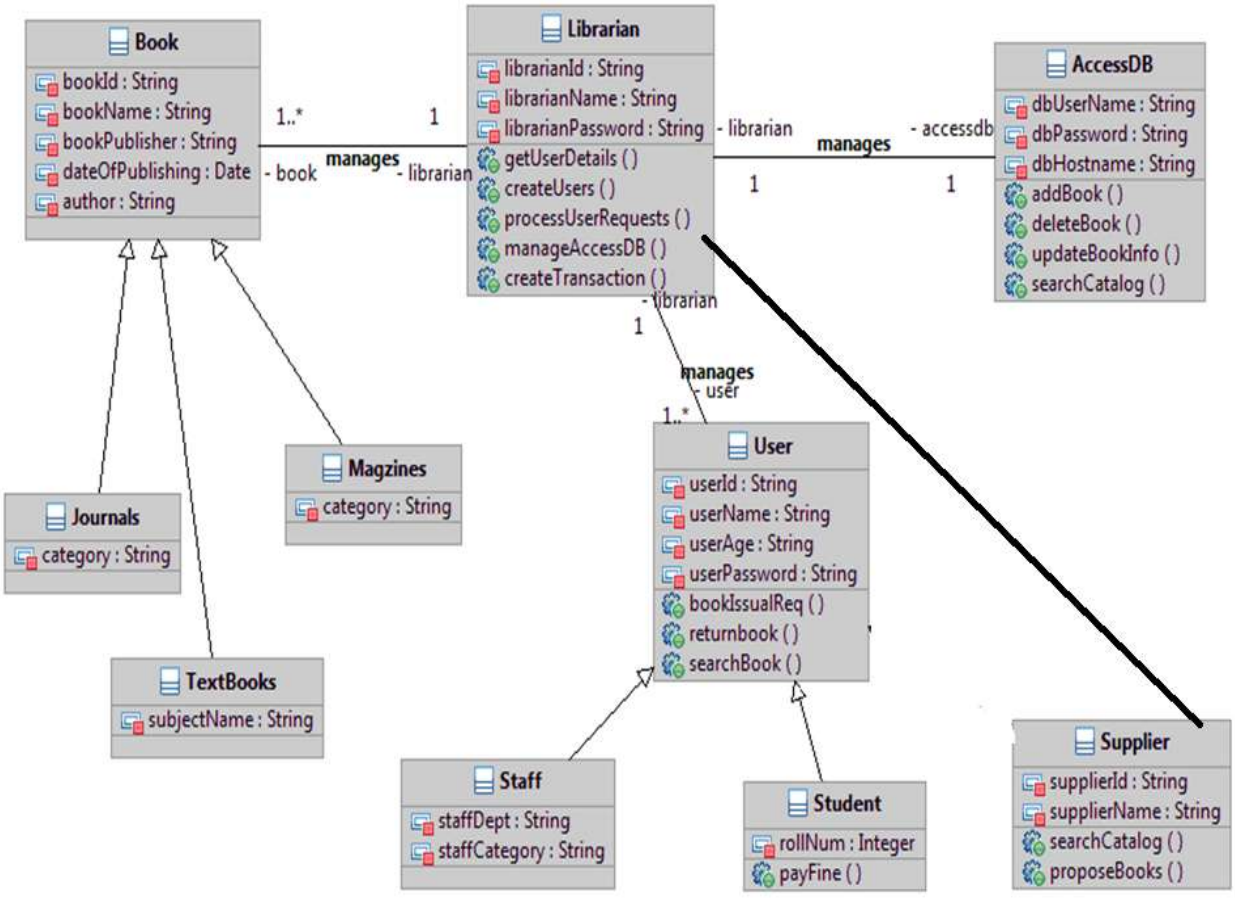
Key Elements of a Class Diagram:

1. **Class**: Represented by a rectangle, a class is a blueprint for objects. It contains:
    - **Attributes**: The data or properties that a class has (e.g., name, age).
    - **Methods**: The functions or operations that a class can perform (e.g., getName(), setAge()).
2. **Relationships**: Class diagrams show how classes are related to each other. Common relationships include:
    - **Association**: A simple relationship between two classes (represented by a solid line). It may include multiplicities, such as "one-to-many" or "many-to-many."
    - **Aggregation**: A special type of association that represents a "whole-part" relationship, where the part can exist independently of the whole (represented by a hollow diamond at the whole class end).
    - **Composition**: A stronger form of aggregation, where the part cannot exist without the whole (represented by a filled diamond at the whole class end).
    - **Inheritance (Generalization)**: Represents an "is-a" relationship where one class inherits from another (represented by a solid line with a hollow triangle at the parent class).
    - **Realization**: Used to show that a class implements an interface (represented by a dashed line with a hollow triangle at the interface).
    - **Dependency**: A weaker relationship where one class depends on another (represented by a dashed line with an arrow).
3. **Visibility**:
    - **Public (+)**: Attributes or methods are accessible by any class.
    - **Private (-)**: Attributes or methods are accessible only within the class itself.
    - **Protected (#)**: Attributes or methods are accessible within the class and its subclasses.
4. **Multiplicity**: Indicates how many instances of a class can be associated with another. For example, a "1" or "0..*" (many) may be used to show how many instances of one class relate to another.

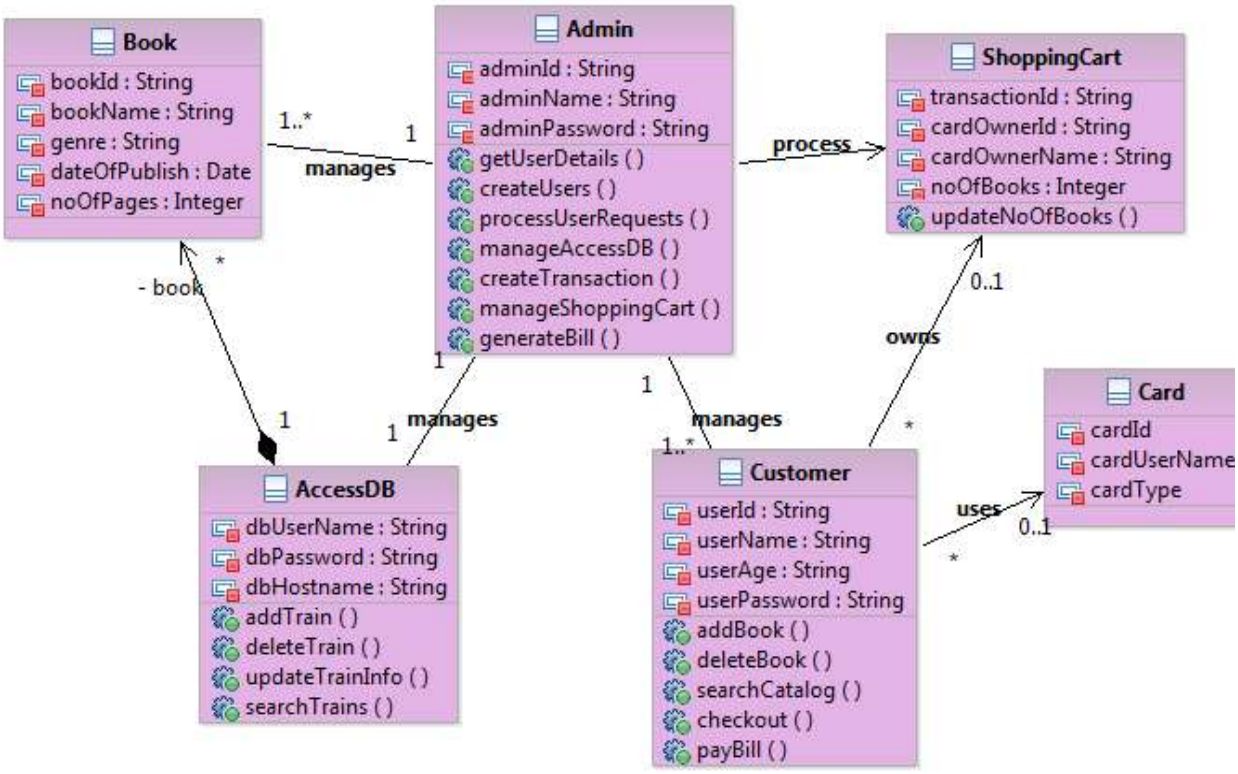
Draw Class Diagram for ATM System.



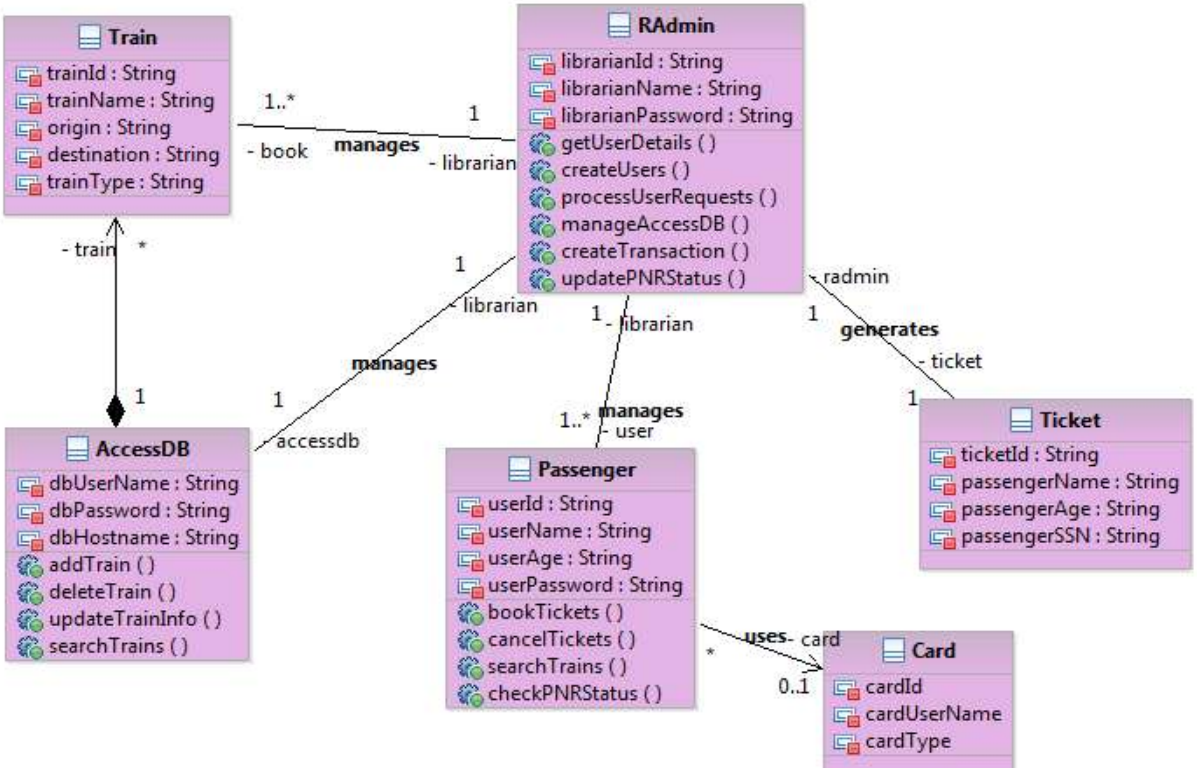
Draw Class Diagram for Library Management System.



Draw Class Diagram for Online Book Shopping.



Draw Class Diagram for Online Train Ticket Reservation System.



3. Sequence Diagram:

A Sequence Diagram is a type of interaction diagram that shows how objects interact with each other in a sequential manner. It focuses on the order of messages exchanged between objects or components in a system to complete a particular scenario or use case.

Key Elements of a Sequence Diagram:

- Objects (Lifelines): Represented by vertical dashed lines, they correspond to the entities involved in the interaction (e.g., users, systems, or components).
- Messages: Horizontal arrows between lifelines represent messages or method calls sent between objects. The arrows are labeled with the operation being called.
- Activation Bars: A vertical rectangle on a lifeline indicating that an object is performing an operation during that period.
- Time: The diagram is ordered vertically, with time progressing from top to bottom.
- Return Messages: Dashed arrows typically represent the return of control or data back from a method call.
- Conditions/Loops: Sequence diagrams can include conditions (alt for alternative paths) or loops (loop for repeating processes).

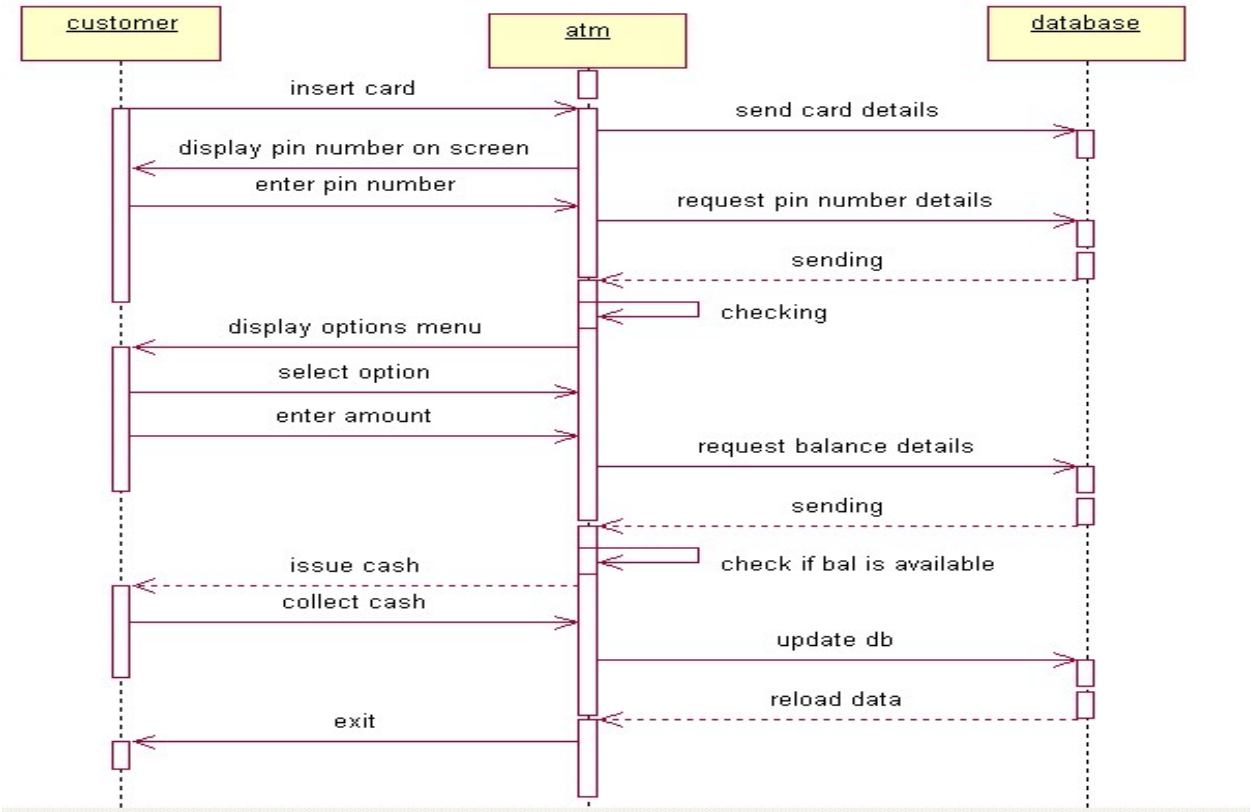
4. Collaboration Diagram (also called Communication Diagram):

A Collaboration Diagram is also an interaction diagram that shows how objects or components interact with each other to achieve a particular goal. It focuses on the structural organization of the objects and the messages exchanged between them, emphasizing the relationships and communication paths rather than the time sequence.

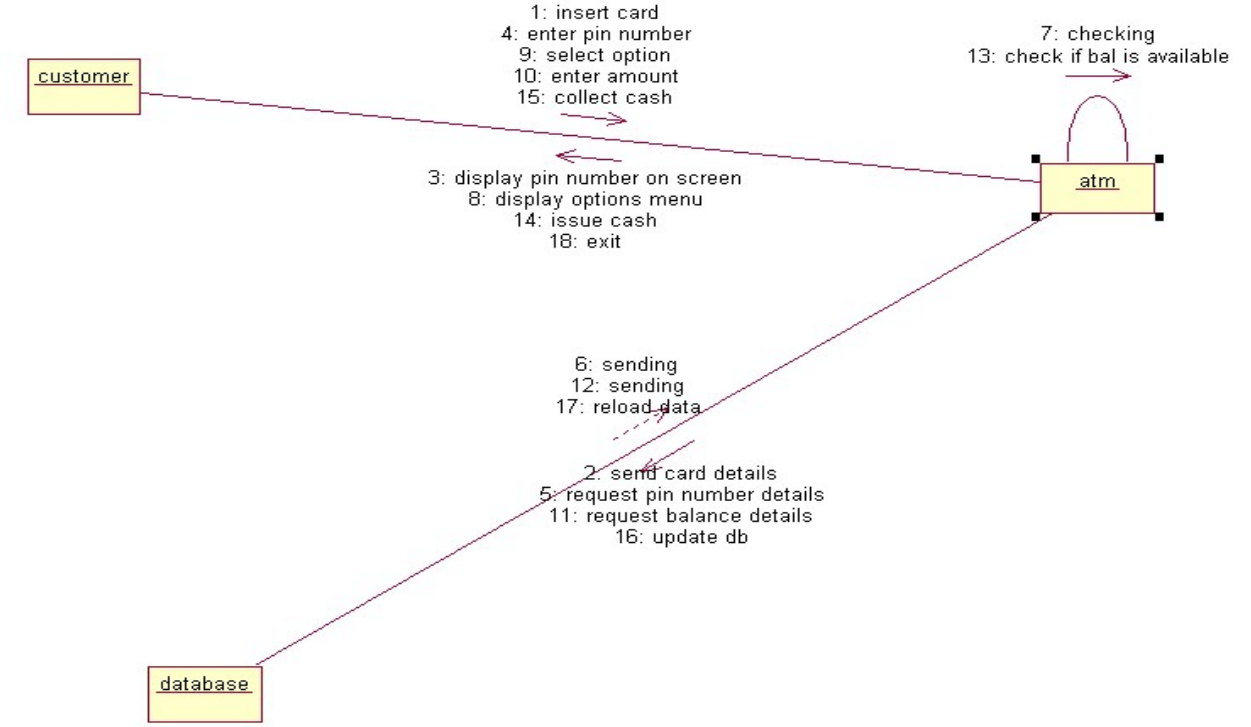
Key Elements of a Collaboration Diagram:

- Objects: Represented by labeled rectangles. They correspond to the entities interacting with each other.
- Messages: Represented by numbered arrows (1, 2, 3, etc.), indicating the order in which the messages are sent between objects.
- Links: Solid lines connecting the objects to show their relationships or associations.
- Message Sequence Numbering: Messages are numbered to show the order in which they occur, typically displayed along the arrow.

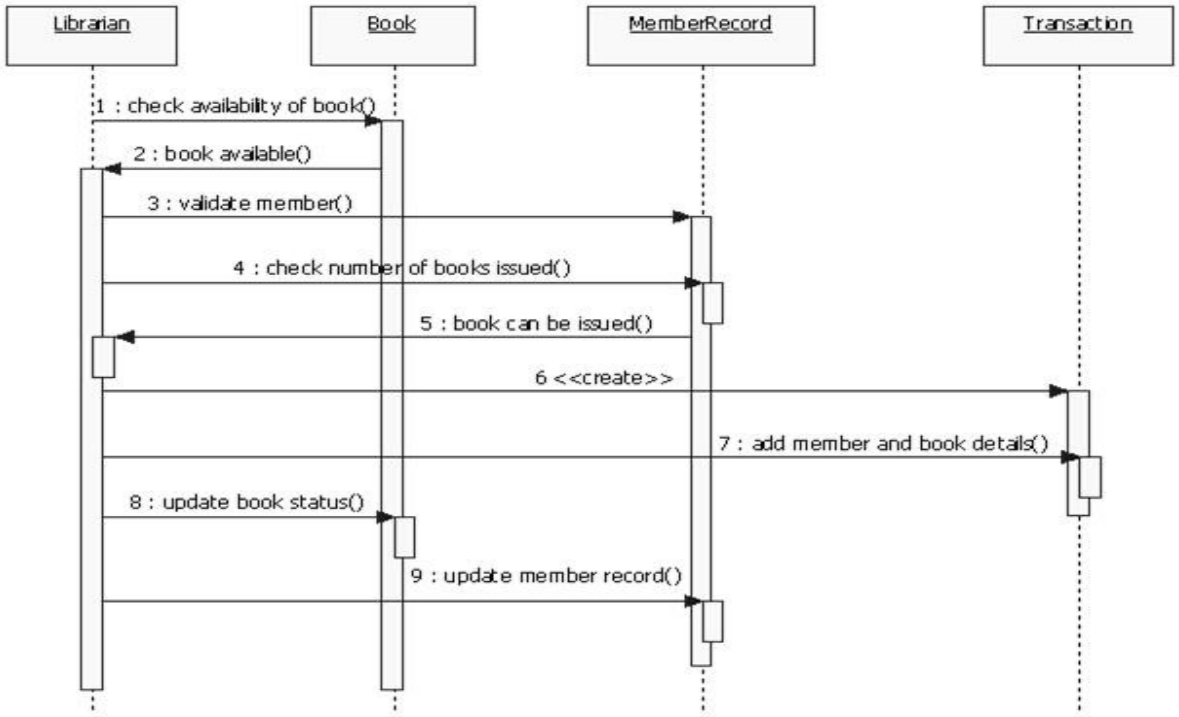
Draw Sequence Diagram for ATM System-Withdraw Amount Operation.



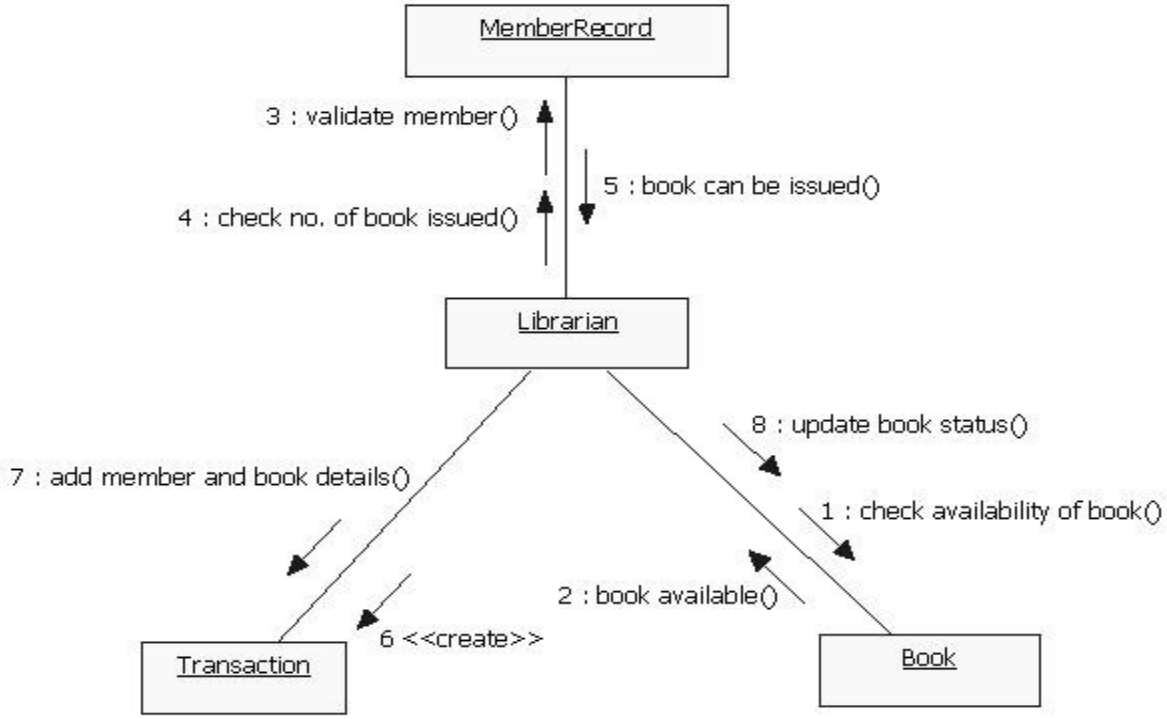
Draw Collaboration Diagram for ATM System-Withdraw Amount Operation.



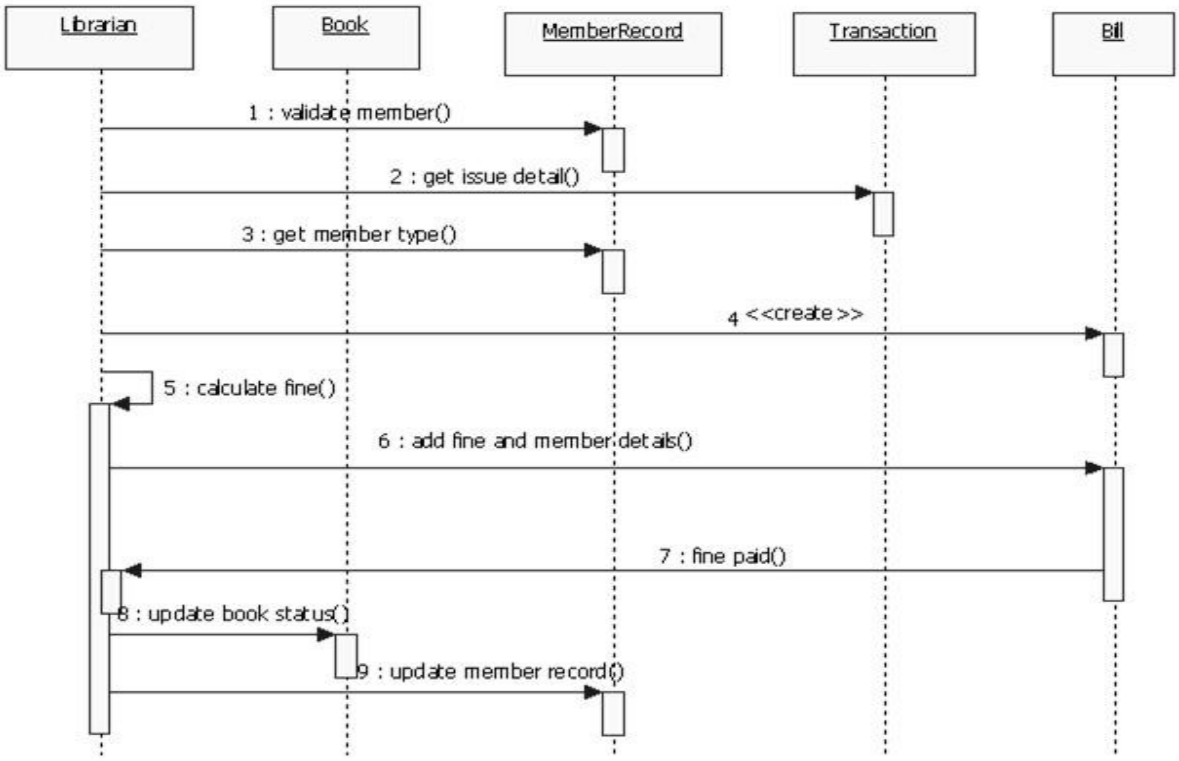
Draw Sequence Diagram for Library Management System-Issue Book Operation.



Draw Collaboration Diagram for Library Management System-Issue Book Operation.



Draw Sequence Diagram for Library Management System-Return Book Operation.



Draw Collaboration Diagram for Library Management System-Return Book Operation.